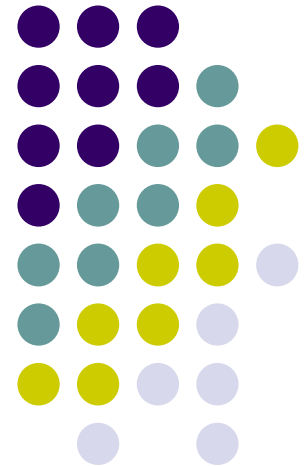
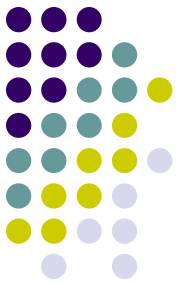


Flutter – Dart Basic

Chapter 3.1

Instructor: Sadullah Karimi
MSc In Computer Science & Engineering



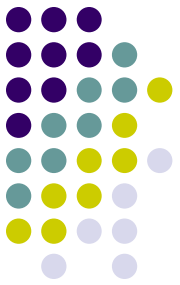


Dart Programming Language

Primarily used with the Flutter Framework
Powerful tool for developing cross-platform
applications



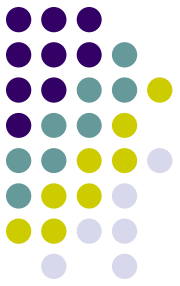
Dart



Dart virtual machine (VM)

- Designed to run code quickly and efficiently
- Involves two phases:
 - Compile time
 - Runtime
- Enables efficient
- Development and high performance execution.

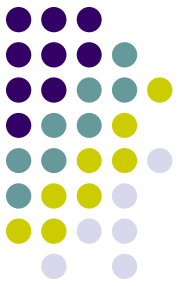
Just-in-Time (JIT) and ahead-of-time(AOT) Compilation



- JIT happens while the program is running
- JIT show changes without restarting the app
- AOT happens before running the program



Garbage collection



**Manages memory with two Strategies:
Young generation and old generation**

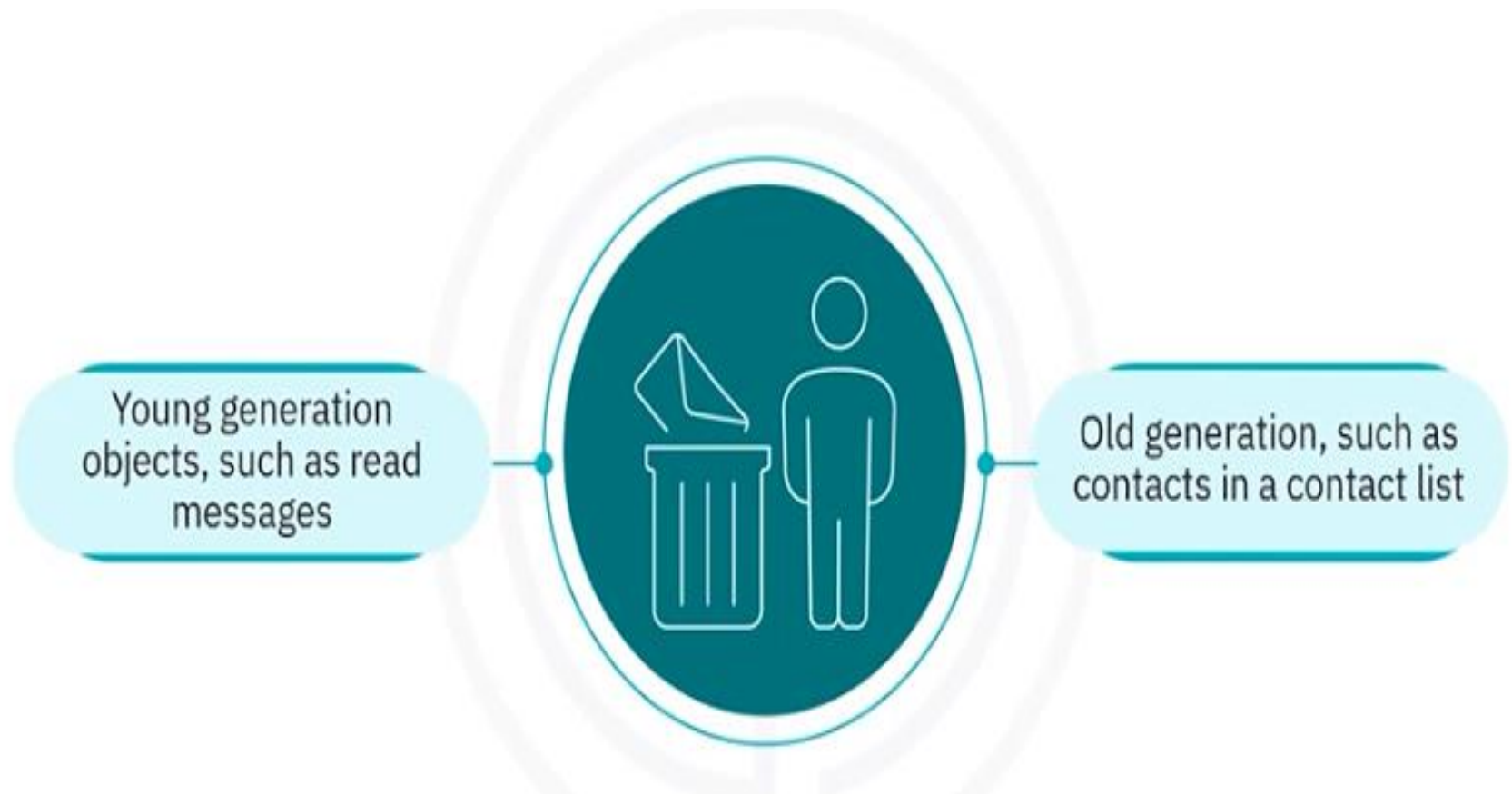
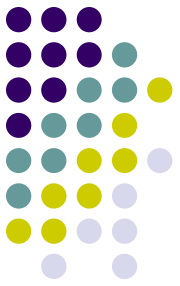


Young generation object are temporary

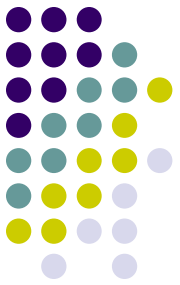


Old generation objects are used for longer periods

Example of Garbage collection



Concurrency



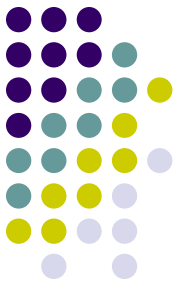
Isolates run at the same time without sharing memory



Async programming uses “async” and “await” to keep code running



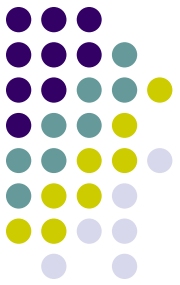
Example of concurrency in dart



Loading data from the internet in the background

Reading large files from the device storage





Core Libraries

dart:core

- Included automatically
- Provides functions and classes

dart:async

- Helps with asynchronous programming
- Includes classes such as Future and Stream

dart:io

- Handles input and output

dart:convert

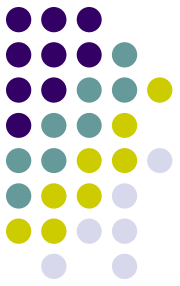
- Helps with converting data

Why is dart the preferred programming language for flutter?

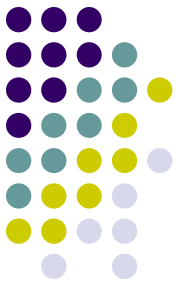


- Compiles efficiently to native code
- Supports cross-platform development
- Support reactive programming for responsive UI
- Uses a syntax similar to JavaScript
- Modern and powerful programming language developed by google
- Backbone of flutter
- Object-oriented principles
- Optimized for UI
- Create smooth animation and transition
- Hot reload feature help with quickly viewing the changes

What are the fundamental concept of Dart programming?



- Everything in a variable is an object
- Every object is an instance of a class
- This includes numbers, functions, and null
- Enable NULL SAFETY to make variable nullable with a question mark
- A nullable integer can be NULL or int
- Uses variables, operators, functions, and classes
- Build any application in flutter
- Structure code effectively for readability and maintenance
- Simplify complex tasks using classes and objects
- Manage network requests with futures and streams

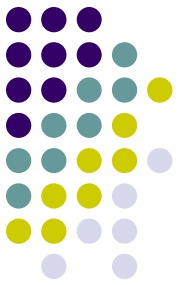


Declare and initialize variables

- In your main.dart file, declare and initialize a few variables with different data types:

```
void main() {  
  int age = 25;  
  double height = 5.9;  
  String name = "John Doe";  
  bool isStudent = true;  
  
  print('Name: $name');  
  print('Age: $age');  
  print('Height: $height');  
  print('Is a Student: $isStudent');  
}
```

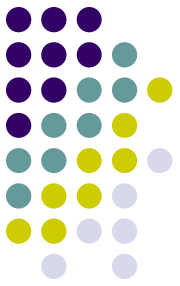
Run Code `dart main.dart`



Explore different data types

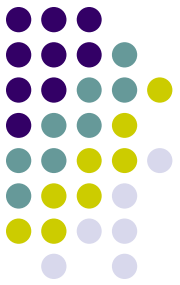
```
void main() {  
    List<String> fruits = ['Apple', 'Banana', 'Cherry'];  
    Map<String, int> scores = {'Alice': 90, 'Bob': 85, 'Charlie': 95};  
    String? nullableString = null;  
  
    print('Fruits: $fruits');  
    print('Score of Alice: ${scores['Alice']}');  
    print('Nullable String: $nullableString');  
}
```

Understand type inference and annotations



```
void main() {  
    var city = 'New York'; // Type inferred as String  
    String country = 'USA'; // Type annotation  
  
    print('City: $city');  
    print('Country: $country');  
}
```

Use constants and final variables



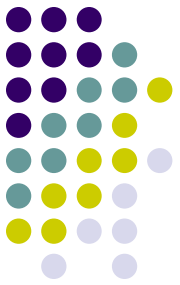
```
void main() {  
    const double pi = 3.14;  
    final DateTime currentTime = DateTime.now();  
  
    print('Pi: $pi');  
    print('Current Time: $currentTime');  
}
```

Pi: 3.14

Current Time: [Current time will vary based on when the code is run]

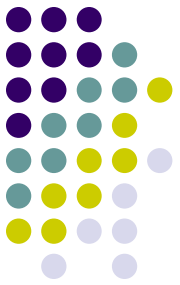
`const double pi = 3.14;`: Declares a constant pi that is set at compile-time.

`final DateTime currentTime = DateTime.now();`: Declares a final variable currentTime that is set at runtime but cannot be changed afterward.



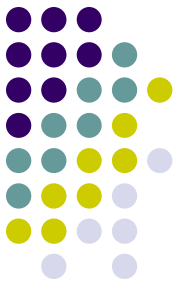
Define and call functions

```
void greet(String name) {  
    print('Hello, $name!');  
}  
  
void main() {  
    greet('Alice');  
}
```



Work with methods in classes

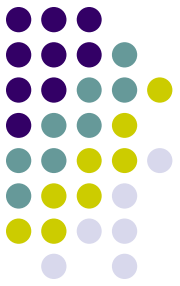
- `class Person {`
- `String name;`
- `int age;`
- `Person(this.name, this.age);`
- `void displayInfo() {`
- `print('Name: $name, Age: $age');`
- `}}`
- `void main() {`
- `Person person = Person('Bob', 30);`
- `person.displayInfo();`
- `}`



What are classes

- Works with real-world entities
- Real-World Entities become objects
- Attribute Represent Data
- Methods Implement behavior

What are class



Method:

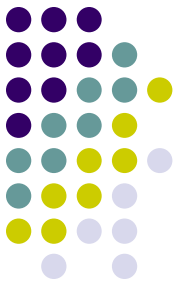
- Accelerate
- Start
- Stop



Properties:

- Payload Capacity
- Cargo bed size
- Towing capacity
- Ground Clearance

Public and Private Property

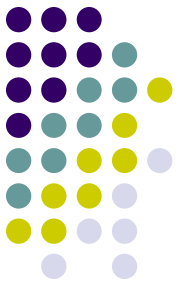


Define object access:

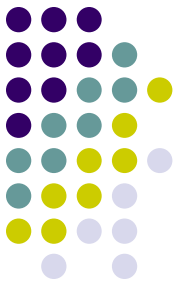
- Public Property: Allows external access.
- Private Property:
- Restrict access



Public and private properties



```
Class Person {  
    //public properties  
    String name;  
    int age;  
    // Private property  
    String _ssn;  
    // Constructor  
    Person(this.name, this.age, This._ssn);  
    //method  
    Void DisplayInfo(){  
        Print('Name: $name, Age: $age, SSN: $_ssn');} }  
    Void main(){  
        Person p1 = new Person('ahmad', 21,'123-45-6789');  
        p1.DisplayInfo(); } // output Name: ahmad, Age: 30, SSN: 123-45-6789
```

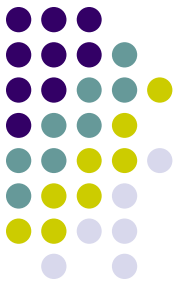


Defining method in classes

- Objects are lifeless without methods
- Method define an object's behavior



shutterstock.com · 2362585547

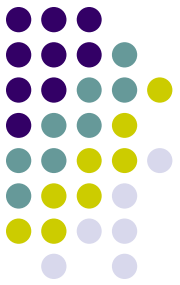


Defining method in class

```
Class Person{
String name;
Int age;
Person(this.name, this.age);
void describe(){
Print('I am $name and I am $age years old.');
```

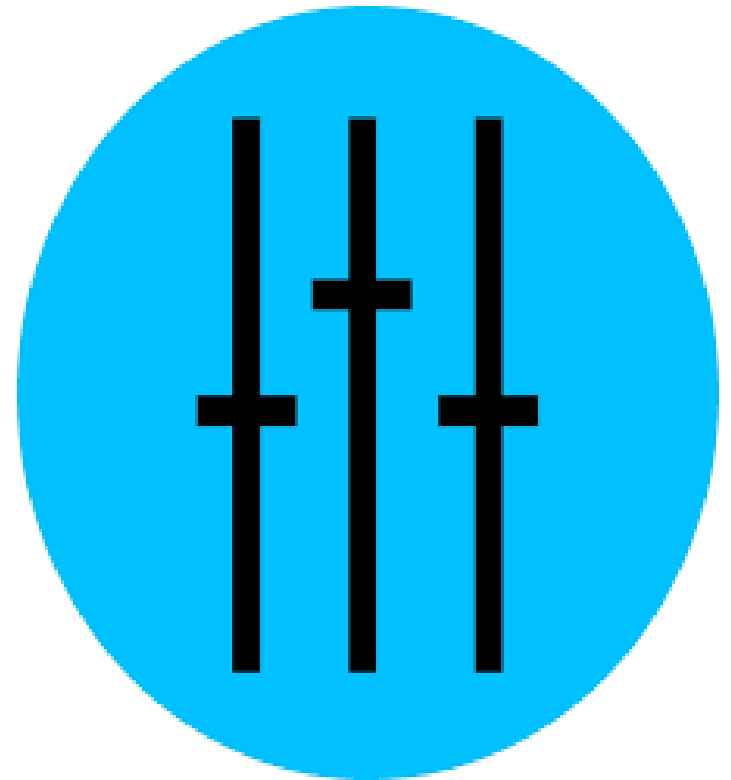


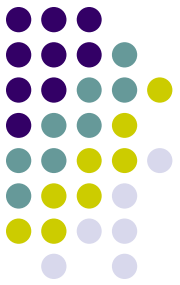
```
}
Int yearsUntilRetirement(){
    return 65 – age;
void main(){
Person p = Person('Alice', 30);
P.describe(); // output: I am Alice and I am 30 years old.
Print(p.yearsUntillRetirement()); // output: 35
```



Getters and Setters

- Control Access to an object's properties
- Getters perform calculations or formatting
- Setters validate input or trigger actions



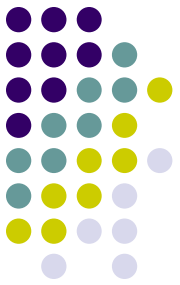


Getters Example

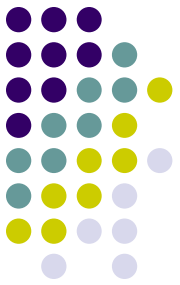
```
Class Circle {  
    double radius;  
    Circle(this.radius);  
    double get area => 3.14 * radius * radius;  
}
```

```
void main() {  
    Circle c = Circle(5);  
    Print(c.area); // output: 78.5  
}
```

Setters example



```
Class Rectangle {  
    double _width = 0;  
    double _height = 0;  
Void set width(double value) {  
    If( value > 0 ) {  
        _width = value;  
    }  
Void set height(double value) {  
    If (value > 0) {  
        _height = value; }  
Double get area => _width * _height;  
Void main() { Rectangle r = Rectangle();  
r.Width = 10;  
r.Height = 20;  
Print(r.area); } // out put 200
```



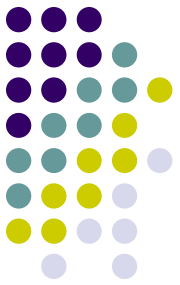
Static methods

Static methods:

- Do not belong to object instances.
- Are used for utility function
- Do not require object access

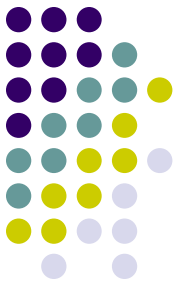
Example:

- Utility Conversion
- Mathematical Calculations.



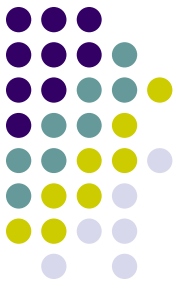
Static Methods

```
Class Math {  
    Static double pi = 3.14;  
  
    Static int square(int x) {  
        Return x * x;  
    }  
  
    Void main(){  
        Print(Math.pi); // output: 3.14  
        Print(Math.square(4)); //output: 16  
    }  
}
```



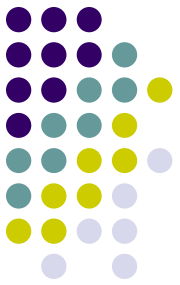
Anonymous functions

- Ideal for short operations
- Offers flexibility in various scenarios
- Promotes clean code for complex code;



Anonymous functions

- `Var list = ['apples', 'bananas', 'oranges'];`
- `List.forEach((item) {`
- `Print(item);`
- `});`
- Output:
- `// apples`
- `// bananas`
- `// oranges`

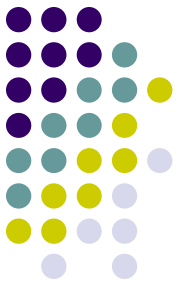


Encapsulation

- World's most valuable jewelry.
- Safely guarded key
- Treasure chest = Class's object
- Key = Getters and Setters

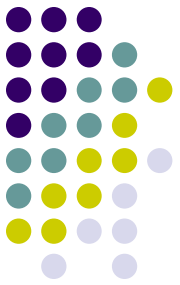


Encapsulation



```
Class BankAccount {  
    String _accountNumber;  
    double _balance;  
    // Constructor  
    BanksAccount(this._accountNum  
ber, this._balance);  
    // Getter  
    double get balance => _balance;  
    // Setter  
    void deposit( double amount) {  
        if( amount > 0) {  
            _balance += amount; } }  
}
```

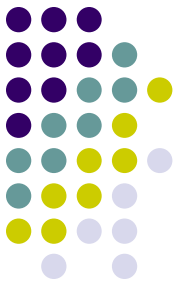
```
void withdraw( double amount) {  
    If (amount <= _balance) {  
        _balance -= amount;  
    }  
    Else [  
        Print(' Insufficient found'); } }  
  
void main(){  
    BlankAcount account = new  
    BlankAcount('123456789',  
    1000.0);  
    Print('Balance:  
    \${account.balance}');  
    // output : Balance: 1000.0
```



Inheritance

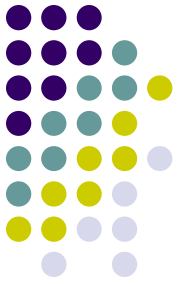
- Child class inherits from parent class
- Animal is a parent class
- Dog, Cat, and Horse are child classes
- Sleep and eating are inherited properties
- Breed or lifespan are child properties
- Inheritance:
 - Helps organize complex code
 - Establishes relationships between parent and child classes.





Inheritance

```
Class Animal {  
    void eat() {  
        print(' Animal is eating');  
    }  
}  
Class Dog extends Animal {  
    void bark() {  
        print(' Dog is barking');  
    }  
}  
void main() {  
    Dog dog = new Dog();  
    dog.eat(); // output: Animal is eating  
    dog.bark(); } // output: Dog is barking
```



Thank You